

换句话说由于你无法使用 `virtual` 函数从 `base classes` 向下调用，在构造期间，你可以藉由“令 `derived classes` 将必要的构造信息向上传递至 `base class` 构造函数”替换之而加以弥补。

请注意本例之 `BuyTransaction` 内的 `private static` 函数 `createLogString` 的运用。是的，比起在成员初值列（`member initialization list`）内给予 `base class` 所需数据，利用辅助函数创建一个值传给 `base class` 构造函数往往比较方便（也比较可读）。令此函数为 `static`，也就不可能意外指向“初期未成熟之 `BuyTransaction` 对象内尚未初始化的成员变量”。这很重要，正是因为“那些成员变量处于未定义状态”，所以“在 `base class` 构造和析构期间调用的 `virtual` 函数不可下降至 `derived classes`”。

请记住

- 在构造和析构期间不要调用 `virtual` 函数，因为这类调用从不下降至 `derived class`（比起当前执行构造函数和析构函数的那层）。

条款 10：令 `operator=` 返回一个 *reference to *this*

Have assignment operators return a reference to **this*.

关于赋值，有趣的是你可以把它们写成连锁形式：

```
int x, y, z;  
x = y = z = 15;           //赋值连锁形式
```

同样有趣的是，赋值采用右结合律，所以上述连锁赋值被解析为：

```
x = (y = (z = 15));
```

这里 15 先被赋值给 `z`，然后其结果（更新后的 `z`）再被赋值给 `y`，然后其结果（更新后的 `y`）再被赋值给 `x`。

为了实现“连锁赋值”，赋值操作符必须返回一个 `reference` 指向操作符的左侧实参。这是你为 `classes` 实现赋值操作符时应该遵循的协议：

```
class Widget {  
public:  
    ...
```

```

Widget& operator=(const Widget& rhs)           //返回类型是个 reference,
{                                              // 指向当前对象。
    ...
    return* this;                             //返回左侧对象
}
...
};

```

这个协议不仅适用于以上的标准赋值形式，也适用于所有赋值相关运算，例如：

```

class Widget {
public:
    ...
    Widget& operator+=(const Widget& rhs)      //这个协议适用于
    {                                          // +=, -=, *=, 等等。
        ...
        return *this;
    }
    Widget& operator=(int rhs)                //此函数也适用，即使
    {                                          // 此一操作符的参数类型
        ...                                  // 不符协定。
        return *this;
    }
    ...
};

```

注意，这只是个协议，并无强制性。如果不遵循它，代码一样可通过编译。然而这份协议被所有内置类型和标准程序库提供的类型如 `string`, `vector`, `complex`, `tr1::shared_ptr` 或即将提供的类型（见条款 54）共同遵守。因此除非你有一个标新立异的好理由，不然还是随众吧。

请记住

- 令赋值 (*assignment*) 操作符返回一个 `reference to *this`。

条款 11: 在 operator= 中处理 “自我赋值”

Handle assignment to self in operator=.

“自我赋值”发生在对象被赋值给自己时：

```

class Widget { ... };
Widget w;
...
w = w; //赋值给自己

```

这看起来有点愚蠢，但它合法，所以不要认定客户绝不会那么做。此外赋值动